Big Data Management
Building a Big Data Architecture

Project guidelines (P1): contextualization, data ingestion and Landing Zone

1. Pipeline architecture

We explicitly ask you to define, in this first deliverable, the **architecture** of your processing pipeline. That is, a high level diagram to showcase the flow of data. The goal is to put yourself in the shoes of a data engineer who has to come up with an entire workflow before implementing it. Given that the Trusted and Exploitation Zones will not be presented in P1, you can define them as black boxes to be later instantiated. Nonetheless, you should already know what you want to do with the data, so you can already present the processes that will exploit the data that you manage (i.e. you have to include the data consumption stage).

This diagram is crucial for you to organize the work to be developed. Hence, this is the first thing you should do before writing any code. **At the end of the document we provide a high-level example design of a complex pipeline** to help you in the definition of the architecture. It is highly encouraged to take a look at it now to get a feel for what we are asking of you. You will understand many of the technologies and concepts of the diagram as you read the document and attend the lectures.

The design you see is quite complex, and you should aim for something similar when defining the architecture. **This is because you do not need to implement everything you present in the design**. You might create a big structure and then decide to focus on specific parts of it; those that seem more feasible. Or, during the development you might realize that you can not implement everything and just do a subset of the processes. The goal, once more, is that from the start you have an idea of where you want to go and how to get there, laying up a sophisticated blueprint that you can later modify ad-hoc. Communication with your supervisor is key to know what to prioritize.

Constraint 1: Architecture diagram. You need to design a high-level diagram of your pipeline (you can take the one we propose as an example). It should include all the main stages of the architecture, how the data flows and, ideally, any technologies you use alongside other relevant aspects.

2. Stages (P1)

2.1 Contextualize the problem

Domain and value

Before embarking on a big data project, it is essential to clearly **define the domain** you will be working with. You are free to choose any domain that allows you to align your project with your interests (e.g. healthcare, finance, IoT), as long as it contributes to the development of robust data management pipelines. That is, select a subject that excites you, but also presents meaningful challenges to solve by exploiting data assets.

Once the general context of your system is established, the next crucial step is to **determine the business value** it aims to deliver. That is, what do you want to do with the

data once it has been processed? What problem(s) does your pipeline solve? What insights or improvements does it provide? Defining a clear and measurable objective is fundamental for designing an effective architecture. Additionally, you need to define **how to provide this value**. Are you going to train ML models? Are these going to require interacting with Deep Learning architectures? Are you going to monitor real-time behavior?

Ideally, you would analyze the selected market, detect potential needs and define a solution for the identified problem. Nonetheless, we acknowledge that you do not have the resources to perform such analysis, so it is enough to define a solid project that provides some type of value.

Important note: BDM is not a modelling / analysis course. Hence we will not evaluate the quality or correctness of the tasks you implement when exploiting the data (more on this in the second part, do not worry about it at this point). The goal is not to obtain high-quality insights, but rather to properly define the data flows and the interactions. As such, considering what you aim to achieve will help to structure your system.

Data sources and datasets

Once you have contextualized your project, you have to **select the data sources** to extract the data from. Since you most probably do not have access to data sources, you may want to explore popular repositories to find interesting data (e.g. Kaggle, UCI Machine Learning Repository, AWS Open Data, etc). Here it is important to understand what type of data you will be processing. Broadly, data falls into three categories:

- **Structured Data**: highly organized and schema-based, making it easily searchable and processable. It is typically stored in relational databases, where data is arranged in tables with clearly defined rows and columns. Common formats include SQL databases or CSV files.
- **Semi-Structured Data**: has some level of organization but does not conform strictly to predefined schemas. Unlike structured data, it can accommodate changing attributes and hierarchical relationships, but this flexibility can present challenges in consistency, indexing and querying. It is often stored in adaptable formats such as JSON or XML.
- **Unstructured Data**: lacks a predefined schema and does not fit neatly into traditional databases. It includes raw, complex data types such as images, videos, audio recordings, emails, PDFs, and social media content.

Each category of data is used to capture different domains, serves different purposes and requires specialized tools and techniques for efficient storage, retrieval and analysis. Hence, before starting the data collection process, you **need to understand what type of data you need** for the specific problem you defined.

Tabular (i.e. structured) data is the most popular variant due to being straightforward to work with. However, you have probably worked with it in several other courses (and will do so in several more). Hence, in this course we **ask that you work with semi-structured and unstructured data (in addition to tabular data)**. During the course you will see technologies that will allow you to efficiently store and process these formats, so you can get some hands-on experience on handling non-tabular data.

Additional considerations

Synthetic data usage: we acknowledge that you might have a certain idea to implement, but not the data to do so, especially if the envisioned workflow is complex. Given that the priority in this course is the adequate definition of processing pipelines, we accept the employment of synthetic data that you have generated, provided that it helps you to implement a more interesting project. If you opt for synthetic data, provide the generation scripts you employ.

Data volume: the *B* in BDM stands for *Big*. Hence, in an ideal scenario, the data you employ should be of a decently large size (e.g. not a couple of CSV files of a few KBs) to truly showcase the capabilities of your pipeline. Similarly as with the previous consideration, we acknowledge that finding high amounts of data for your project is not always possible. Hence, it is not a hard constraint to use a lot of data, but doing so will positively influence the evaluation.

Constraint 2: Contextualization. Provide an adequate contextualization of the project, defining the domain, the task to tackle and the value you provide.

Constraint 3: Data selection. Identify what type of data you need. Then, search on popular repositories to obtain the required assets (or create them). Provide a brief explanation of each dataset. Note that you must include structured, semi-structured and unstructured data.

Factors that will **positively contribute** towards the grade:
- Employing several data sources (e.g. fetching data from different APIs) and/or several datasets (e.g. given a social media API, that you fetch both user profiles and posts).
- Great contextualization of the project (i.e. well thought-out domain and business value selection).
- Using “Big” Data.

2.2 Data ingestion

Data ingestion is the first and foundational step in any big data architecture. It involves acquiring raw data from various sources and transferring it into a system where it can be processed, stored and analyzed.

Batch vs. Streaming

One of the most critical decisions data engineers must make when designing data ingestion pipelines is determining the ingestion **frequency**. Depending on the rate at which data is fed into the system, ingestion can be broadly categorized in two:

- **Batch Ingestion:** data is collected in fixed-size chunks (batches), often at scheduled time intervals (e.g. hourly, daily). That is, data is fetched from APIs, databases or other sources at specific moments, and the extracted instances (the batch) tends to follow some criteria (e.g. datasets uploaded in the last day). This

approach is suitable for scenarios where data processing is not immediate and we can afford to query it at specific moments and store it before using it.
- **Streaming (Real-Time) Ingestion:** data is continuously ingested and processed as it arrives, enabling real-time analytics and applications. Streaming ingestion provides the ability to monitor and react to data as it flows, facilitating decisions and actions based on the most up-to-date information. However, streaming ingestion requires a more sophisticated infrastructure to ensure that data is processed and delivered without delay.

Data paths

Streaming has become a necessity in recent years due to the development of systems that provide a constant influx of data (e.g. sensor data, social media feeds, financial transactions), which can be used to monitor certain behaviors in real-time. However, this complicates the management aspect, as it requires a robust infrastructure and continuous resources. Moreover, a large quantity of data does not need to be processed continuously, so batch processing is still fundamental for big data architectures.

In most complex pipelines, there exists a **combination of both approaches**, each addressing different data sources and handling data designed to fulfill different goals. That is, different data *paths* are defined:
- **Hot Path** (Real-Time Processing): This path is dedicated to handling time-sensitive data that requires immediate action (e.g. detecting fraud in financial transactions or triggering alerts), ensuring that data is processed and acted upon as quickly as possible. It typically involves high-throughput systems and low-latency processing frameworks that prioritize speed and reliability.
- **Cold Path** (Batch Processing): This approach handles data that do not need to be processed immediately, being accessed long after it has been obtained. It is often obtained through scheduled batch jobs. This can include processing logs, performing large-scale ETL operations, or running complex analytics.
- **Warm Path** (Near-Real-Time Processing): Positioned between the hot and cold paths, the warm path processes data that is not necessarily urgent but still requires timely analysis. If needed, it can even be stored temporarily for short periods. For example, this could involve monitoring patterns over the past few minutes or hours to adjust operational workflows.

You need to decide which **ingestion policy** works best for your project. If you want to continuously monitor an influx of data to provide real-time processing or updates, you need to set up a streaming ingestion. If that is not relevant for the task you address, you can opt for batch processing. Ideally, you would want to apply both policies to define several processing pipelines. Once that is decided, you need to understand how to apply the selected ingestion pattern to the sources previously selected.

You are already familiar with batch ingestion from other courses, but data streaming is probably a new topic. You will learn about its theoretical aspects during the course. Nonetheless, we encourage you to explore streaming hands-on in the project, as it is an interesting alternative. Setting up a simple streaming environment is not difficult, and you can even generate your own streaming data if you can not find any sources. Both of these can be done by using Apache Kafka, which is the most popular tool for streaming

and the one we recommend. To process the streaming (more on this in the second part of the project, at this point is not needed) you can use Apache Kafka Streams.

Ingestion mechanisms

The most straightforward data ingestion approach is to download files from the sources and manually place them into the pipeline. **This is not acceptable**. We want you to automate, at least, the task of introducing the data into the pipeline. That is, if you download files and store them locally (which we do not suggest either way), you need to implement a mechanism that reads these same files into the pipeline.

Many modern data sources provide APIs that allow programmatic data retrieval, so a more interesting approach would be to use APIs to obtain your data, which requires scheduling the API calls. In this way you are directly interacting with the sources, which defines a more scalable solution. Logically, other methodologies such as database connection or web scraping are equally valid.

Constraint 4: Data ingestion design. Select an ingestion strategy (batch, streaming or both) for each of the data sources you have selected. This should be in accordance with the type of processing (i.e. path) that each data should undergo to fulfill the desired goals.

Constraint 5: Data ingestion implementation. Choose appropriate tools to automate / manage ingestion and implement your ingestion tasks.

Factors that will **positively contribute** towards the grade:
- Implementing both batch and stream processing pipelines.
- Defining several processing paths.
- Ingestion systems that communicate with external sources (i.e. that do not read files from your system, but rather that access non-local environments).
- *Scheduling* data ingestion tasks (e.g. read batch data from an API every hour). More on this further down the document.

2.3 Landing zone

The next step in the pipeline is to store the raw data you have ingested in a single repository, which we define as the Landing Zone. That is, **no processing is being applied at this stage; we store the data as it comes**. Having a common repository through which all the data enters into the system provides several benefits, such as:

- Homogeneization of the starting point of your processing paths. That is, the system becomes agnostic to the consulted data sources and how to access them, and starts directly from the data.
- It ensures data availability, traceability and reproducibility, as you can always consult the original data.
- It does not force data to adapt to a given model (read-first, model-later) right away, as opposed to traditional databases (model-first, read-later).

Note that you might **not want to store all the data** that you have ingested. Most of the data that comes from batch ingestion is stored, as its use is not immediate and we can afford to curate it before use (cold path). On the other hand, streaming data might be needed immediately (hot path), so rather than storing it, the data is directly processed to obtain some type of insight. As data exploitation is not yet introduced, if you opt for streaming you only need to set up the data ingestion, storing it wherever you want.

An interesting alternative is to store aggregations over the streaming data for posterior analysis (e.g. average value across a time period). If these aggregates need further refinement, they can be stored in the landing zone and be later processed. The decision on how to handle the different types of data you ingest depends on your goals, and it is ultimately a design choice on your part.

Data lakes

These large (as the amount of raw data might be huge) and heterogeneous (as we will have data of many different formats) repositories we are defining here are commonly referred to as **data lakes**. The easiest way to implement a data lake is to map it to your file system. That is, you can create some folders in your PC that will store the data coming from the ingestion mechanisms. Any management that needs to be done can be executed via Python scripts.

This approach is very limited and not inherently distributed, which, as you will see during the initial lectures, is crucial for scaling the processing of data. Traditionally, if you wanted to distribute your data, you had to set up your own cluster of machines and deploy a distribution protocol such as HDFS (you will learn about it during the theory sessions). This was problematic, as you had to acquire the hardware and properly configure the environment, requiring specialized engineers. As a result, the **cloud** was devised as a solution. Technologies such as Amazon S3 or Azure Data Lake Storage automatically distribute the data you send them, thus preventing you from needing to set up your own cluster and handling the configuration.

The main point is that, rather than storing your data locally, you can store the data in a cloud service and benefit from the distribution aspects without requiring specialized knowledge. Nowadays, the cloud has become the de facto option for storing data, and most companies employ cloud services to prevent dealing with the low-level infrastructure that would be required otherwise. This also applies to the level of individuals, as most people use Google Drive or similar services. If exploring the cloud seems interesting to you, you can implement your pipeline in one of these services. However, this is not required in BDM, as the focus is to learn the best practices of managing data in modern environments, not interacting with the cloud. Hence, **we recommend that you implement your processing pipelines locally**.

While a local setup is not distributed, modern tools make migrating to the cloud straightforward. In most cases, it only requires changing the data location from your local system to a remote online endpoint. That is, take this project as a staging area where you test the data workflows locally before potentially deploying them in a production environment.

Object stores

We mentioned that storing the data in your folder is the simplest way of doing it. However, we encourage you to use a slightly more complex approach that provides a lot of benefits: using an **object store**. Object storages are a storage architecture that manages data as discrete units called objects, rather than as files. That is, "elements" (e.g. videos, images, etc.) are stored in its raw format alongside internal metadata, which makes it easy to locate and retrieve. This approach allows object storage to scale massively, store unstructured data efficiently and support features like versioning or replication. Essentially, it is a simple and effective way to store everything.

Object stores such as MinIO employ *buckets* rather than folders, which are a slightly more complex iteration of a folder. One of the advantages of MinIO is that it uses the same API as Amazon S3 (the object store used in AWS), meaning that if you use MinIO locally and then want to deploy your pipeline in AWS, you can do it even more easily, as the way data is accessed is the same; you only need to indicate where the bucket is located (either in your PC or in the cloud). Moreover, it offers a web UI from which you can observe the data that has been stored.

Data Lakehouses and Open Table Formats

Following what we have seen so far, the implementation of the Landing Zone (which follows the data lake paradigm) should consist of a series of buckets deployed in your local system. However, it is worth noting that in recent years a new storage framework has emerged: the **data lakehouse**. These combine the storage philosophy of data lakes (large, heterogeneous repositories) with features designed to create an engineering experience similar to a data warehouse (e.g. robust table and schema support, incremental updates and deletes, etc.). Hence, they are a great alternative to implement the Landing Zone (and also the following stages).

So, the question arises: how is a data lakehouse implemented? The first step (corresponding to the data lake part) has already been mentioned: storing the data in object stores, which are way more flexible than specialized databases. The second, crucial step is the addition of a layer to handle the data warehouse features. This is often implemented via **Open Table Formats** (OTFs), the three main alternatives being Apache Iceberg, Databricks Delta Lake, and Apache Hudi.

OTFs are technologies that, essentially, manage metadata. By doing so, they provide a series of functionalities traditionally associated with data warehouses: version tracking, ACID transactions, schema evolution, rollback capabilities, etc. More specifically, OTFs are tools that can be invoked in your code and their goal is to create and interact with the metadata they generate. For example, if you had a *my_table.csv* file, by using Delta Lake you would get a folder that looks like this:

```
/my_table/
    data/
        part-0001.parquet
         part-0002.parquet
    metadata/
        00001.metadata.json
```

First, the raw data is transformed to *parquet*, a specialized format that compresses the data (reducing its weight), while keeping all the information (you will see parquet and other data formats during the course). Then, several metadata files are created, which store the necessary properties for subsequent management. Hence, rather than storing *my_table.csv* in the object store, you would store this folder structure and, when the data needs to be queried or updated, the metadata can streamline the process.

Note that we have just presented what to expect when using an OTF, as exploring the possibilities of what they offer remains your job. Also note that transforming the data to parquet should mainly be done for structured data. Images, videos and similar data types should be left in their native format, as we do not need to query them as we would a tabular dataset. Nonetheless, OTFs can still be applied to generate metadata artifacts that can ease certain management aspects.

We encourage the use of both object storages and OTFs for the implementation of the Landing Zone. In particular, we recommend MinIO and Delta Lake.

Data organization

Establishing a **well-structured organization** (both for the Landing Zone as well as the overall pipeline) is crucial to facilitate efficient data management and future processing. If this is not done, rather than a data lake you have a *data swamp*.

A simple, yet effective, strategy to provide some structure includes organizing files based on logical properties. If you are using an object store you can create a bucket that will contain all the data of the Landing Zone. Then, you can organize the bucket into folders to accommodate the data ingestion. For example, you can create a folder for every source you employ and store its data there, or for every relevant domain that you have. Additionally, you should implement a standardized naming convention across all files to enhance consistency and make retrieval seamless.

Beyond these basic practices, more advanced data management techniques can be employed to improve the processing of data. We encourage you to explore what OTFs allow and include some of its aspects into the pipeline. Alternatively, you can extend the pipeline by including "subzones", like a *Temporal Landing* Zone in which you initially store the raw data (with no structure) for a period of time before it is deleted and moved to a *Persistent Landing Zone,* in which the data is further structured and kept indefinitely.

By enforcing these approaches, the first storage layer not only ensures data integrity but also lays the foundation for a scalable and well-governed data ecosystem.

Constraint 6: Landing Zone design. First, select an appropriate storage solution based on the nature of your data (e.g. data lakehouse). Second, define an organizational structure for storing raw data (e.g. different folders/buckets, naming convention, data partitioning). Third, indicate the series of technologies and methods to be employed.

Constraint 7: Landing Zone implementation. Choose appropriate tools to implement the Landing Zone, deploy it and store the data that was ingested.

Factors that will **positively contribute** towards the grade:
- Using object storages.
- Defining a data lakehouse.
- In-depth metadata management.

3. Orchestration and containerization

The initial versions of the pipeline are meant to be developed asynchronously. That is, you will first set up the different environments and their respective connections separately, manually testing that the data flows adequately for each specific segment of the architecture. Nonetheless these pipelines are meant to be automated, from the data ingestion to the exploitation of the data. Hence, we ask that, once the stages have been set in place, that **you automate (more precisely, orchestrate) the entire execution**.

Key aspects of workflow orchestration include:

- Task scheduling: Defining when each process should run, whether on a periodic basis (e.g., daily, hourly) or triggered by specific events.
- Dependency management: Ensuring that tasks are executed in the correct order, preventing downstream tasks from running before upstream processes have completed successfully.
- Error handling and retries: Automatically detecting failures, logging errors, and retrying failed tasks as necessary to ensure robustness.

You will study Apache Airflow, an orchestrating tool, later in the course. We encourage you to start experimenting with it now. Alternatively, you can use simple Python scripts to schedule the tasks.

Constraint 8: Pipeline orchestration. Once the implementation for the ingestion and Landing Zone have been completed, you have to orchestrate the execution (i.e. automate the execution of the scripts so you do not have to manually trigger each task). Preferably, this should be done via Apache Airflow, although Python scripts are accepted as an alternative.

Recall that we ask for you to provide access to a repository to get the project implementation from. All the tools we propose can be interacted with programmatically via Python (i.e. you can download a Python library and interact with the tool). However, several of them (e.g. Apache Kafka, Apache Airflow) require the **deployment of some service**. You can do this manually by installing and running the necessary process, but an easier and more practical alternative is to use a containerization tool such as Docker.

These tools allow you to streamline the deployment of services, and are ubiquitous in the current Data Science landscape. Rather than having to follow a complex set of commands to install something, you simply download premade images of the tool and deploy them by executing a couple of commands.

Constraint 9: Containerization. If you employ any tool that requires the deployment of a service, you should do so via a containerization tool.

If you want to extend the development even further, you can also add the following notions into the architecture:

- Error handling and reporting.
- Execution monitoring (processes performance, disk and memory usage, etc.).
- Tools to control code quality.

4. Deliverables

Main deliverable (P1)

The tasks described in this document correspond to the **first deliverable** (max. **12 pages long**), which include:

- **Architecture design**
- Stage 0: project context
- Stage 1: data ingestion
- Stage 2: landing zone

To implement each stage follow the descriptions of Section 2.

Constraint 10. Organization. Clearly organize your code and storage repositories per zones / stages. Each zone should have a folder with its respective scripts (e.g. one folder for the data ingestion and another for the Landing Zone). Clearly separate the scripts per task (if / when required).

The document must contain the following sections:

1. **Instructions**. Your project supervisor should have access to your operations environment (e.g. Github). Instructions on how to access and run the code have to be stated here (first section of the document, in a separate page, without numeration). Note that this section does not count for the maximum number of pages to be used in the document.
2. **Contextualization**. State the domain you have chosen, the problem you are tackling and the value it provides. This section should also include a description of the original data sources, where and how you got them from and what do you expect to do with them.
3. **Architecture design**. Provide the diagram of the pipeline you have designed. Include the appropriate explanations to understand the architecture. The goal is to provide an overview of what you have implemented. The diagram should include: sources, data paths, technologies and organization structures (you can take the diagram on the last page of this document as reference).
4. **Data ingestion**. Describe the design of the data ingestion, that is, how are you ingesting your data and where it is going. Include any relevant explanations about the implementation.
5. **Landing Zone**. Describe the design of the Landing Zone, that is, the selected paradigm (e.g. data lakehouse), the storage mechanism (e.g. object storage) and

how the data is organized. Include any relevant explanations about the implementation.

Follow-up deliverables

There will be two follow-up deliverables in this first part, each before their corresponding follow-up sessions. This means that you will have roughly two weeks to deliver them.

First deliverable:

- Project context (first version),
- Architecture design (first version).

The first recommendation is to fully read the document in order to get a feel for everything that needs to be done and the different concepts that appear. Whatever doubts arise can be discussed in the laboratory session.

No implementation is needed at this point. You should have a clear idea of the domain, business value and datasets encompassing your project. This information should be specified in a first draft of the final document to deliver. Also, and employing this knowledge, present the first version of the architecture design.

The purpose of this initial document is for you to have a good understanding of what you want to implement. It does not have to be the final version, but having a first, fully-fledged instantiation of the design will allow for a smoother communication with the supervisor regarding the project.

Second deliverable:

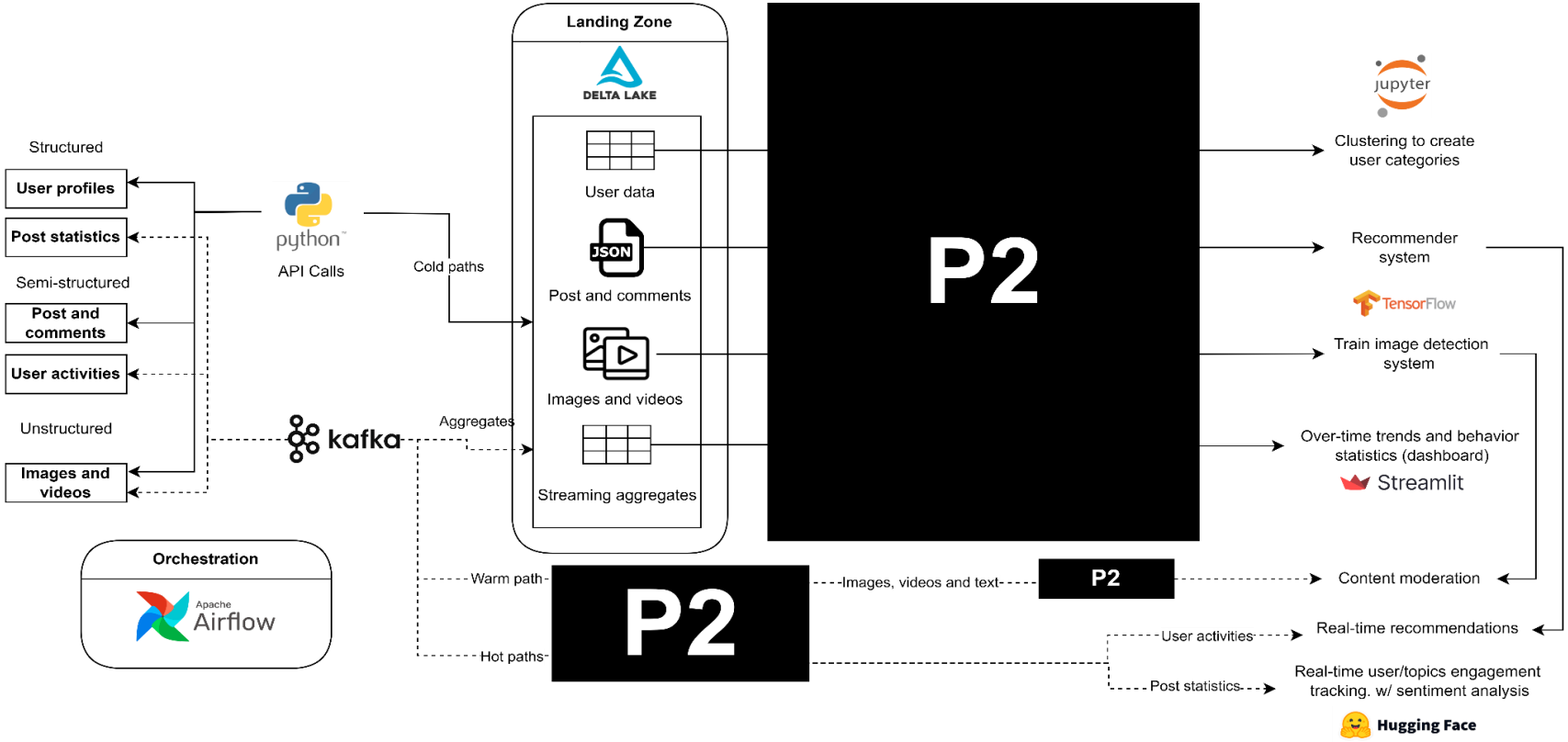
- Project context (updated).
- Architecture design (updated).
- Basic implementation of the data ingestion and the landing zone

First, there should be an updated version of the document provided in the first delivery with the ideas discussed with the supervisor. On the other hand, a link to the project's repository should be provided, with an initial implementation of the data ingestion and landing zone. High-level explanations of these two stages should also be provided in the document.

The goal here is to get familiar with the code and to start setting up the pipeline. The methodology should have been defined and some tests should have been conducted to extract the data from the sources and into the landing zone.

At this point, you will have roughly two more weeks to complete P1. The remaining time should be used to polish the document and prepare the final instantiation of the data ingestion and landing zone.

4. Pipeline design example



This design belongs to a data management architecture that performs social media analysis. Notice the literal black boxes for the aspects not introduced in this first deliverable. You can do something similar or, even better, define on a high level the tasks you plan to do. As for the example we provide, you can observe that different data types are used, different paths are defined and data is being processed in different ways, some of the end tasks even connecting with each other. This type of complex set up is the goal of this project. You can even be more detailed when defining each specific section, including any piece of information you deem relevant.